Computer Science & IT



Database Management System

Transactions and Concurrency control

Lecture No. 10



By- Vishal Sir

Recap of Previous Lecture



- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL
- Topic: Basic time stamp ordering protocol

Slide

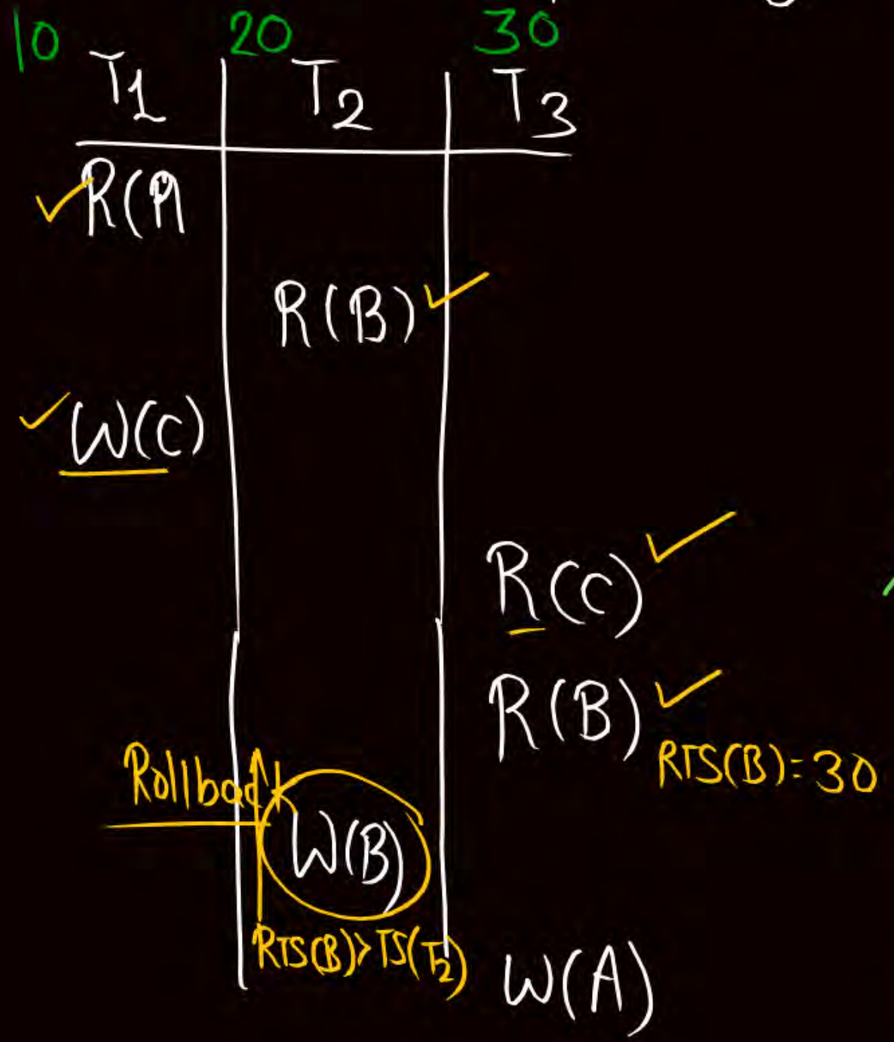
Topics to be Covered



- Topic: Basic time stamp ordering protocol
- Topic: Thomas write time stamp ordering protocol
- Topic: Wait-die protocol for deadlock avoidance
- Topic: Wound-wait protocol for deadlock avoidance

Slide

eg: Consider the following schedule



Which of the following time stamp ordering will allow the schedule to be executed using B.T.S.O.P.

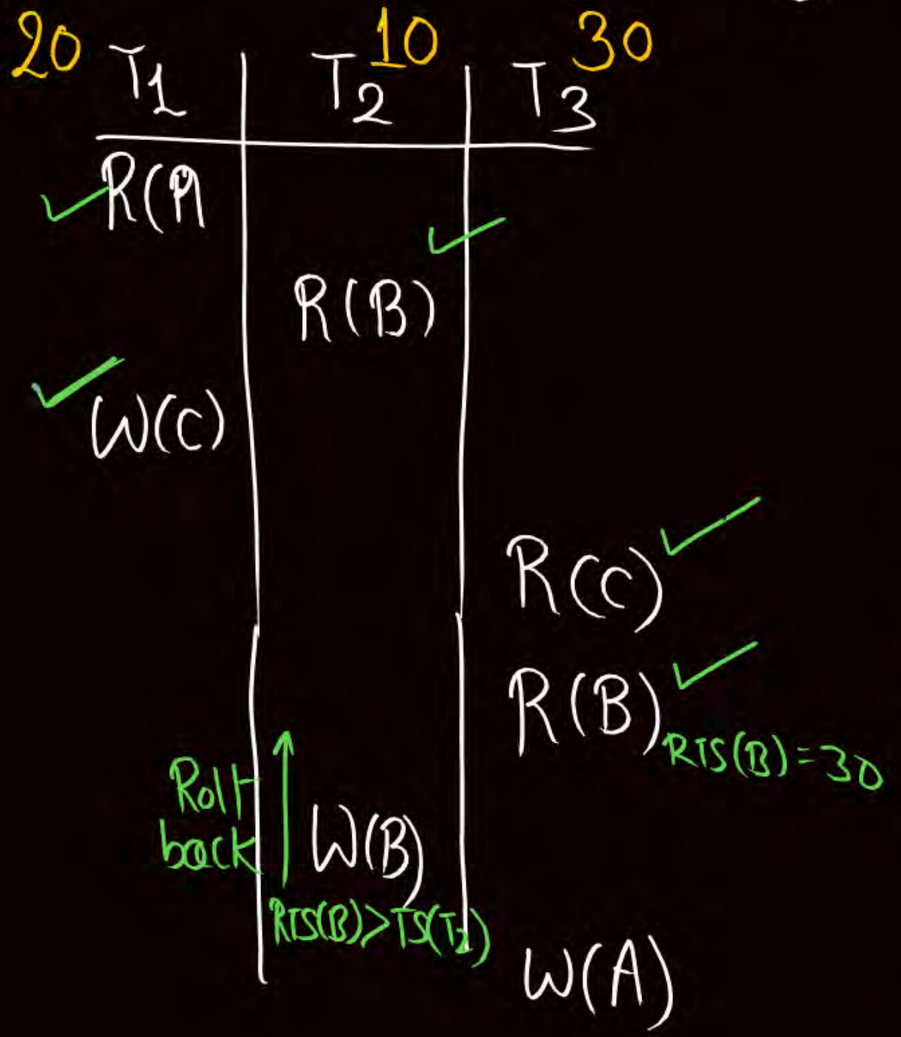
(a) TS(T1, T2, T3) = (10, 20, 30)

(b) TS(T1, T2, T3) = (20, 10, 30)

(c) TS(T1, T2, T3) = (30, 10, 20)

(d) TS(T1, T2, T3) = (10, 30, 20)

eg: Consider the following schedule



Which of the following time stamp ordering will allow the schedule to be executed using B.T.S.O.P.

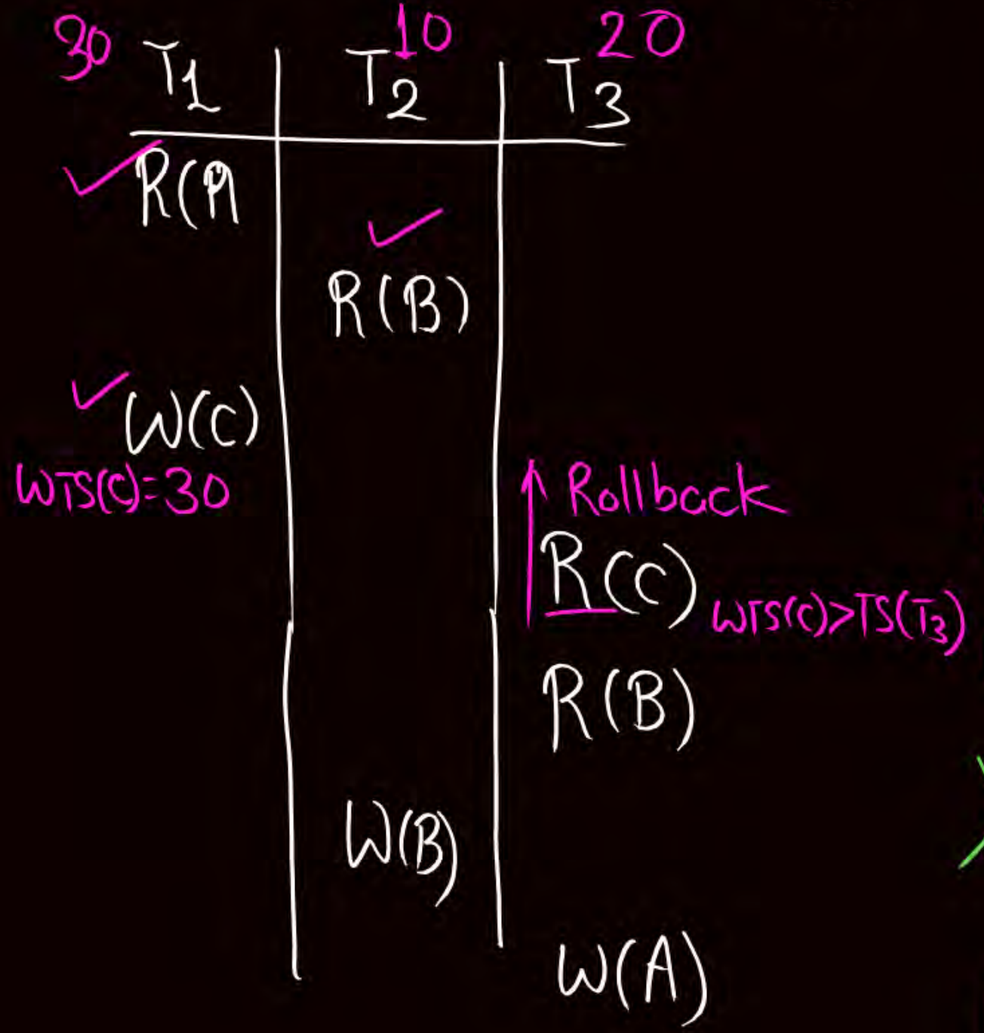
(a) $TS(T_1, T_2, T_3) = (10, 20, 30)$

(b) $TS(T_1, T_2, T_3) = (20, 10, 30)$

(c) $TS(T_1, T_2, T_3) = (30, 10, 20)$

(d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

eg: Consider the following schedule



Which of the following time stamp ordering will allow the schedule to be executed using B.T.S.O.P.

(a) $TS(T_1, T_2, T_3) = (10, 20, 30)$

(b) $TS(T_1, T_2, T_3) = (20, 10, 30)$

(c) $TS(T_1, T_2, T_3) = (30, 10, 20)$

(d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

eg: Consider the following schedule



Which of the following time stamp ordering will allow the schedule to be executed using B.T.S.O.P.

(a) $TS(T_1, T_2, T_3) = (10, 20, 30)$

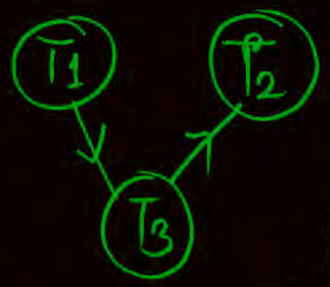
(b) $TS(T_1, T_2, T_3) = (20, 10, 30)$

(c) $TS(T_1, T_2, T_3) = (30, 10, 20)$

(d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

eg: Consider the following schedule

T_1	T_2	T_3
R(A)		
	R(B)	
W(C)		
		R(C)
		R(B)
	W(B)	
		W(A)



Topological order
$= T_1 \rightarrow T_3 \rightarrow T_2$

For this schedule to be allowed by BTSOP

$TS(T_1) < TS(T_3) < TS(T_2)$

Which of the following time stamp ordering will allow the schedule to be executed using B.T.S.O.P.

(a) $TS(T_1, T_2, T_3) = (10, 20, 30)$

(b) $TS(T_1, T_2, T_3) = (20, 10, 30)$

(c) $TS(T_1, T_2, T_3) = (30, 10, 20)$

(d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

Note: ① A schedule is allowed to execute using B.T.S.O.P. if schedule is conflict serializable schedule, and Conflict equivalent serial schedule is based on time stamp ordering of transactions.

② If schedule is conflict serializable schedule but Conflict equivalent serial schedule is not based on Time stamp ordering of the transactions, then schedule is not allowed by B.T.S.O.P.

③ If schedule is not a conflict serializable schedule then it is never allowed to execute using B.T.S.O.P. for any time stamp ordering of transactions.

Note:- ④ B.T.S.O.P. Covers only Conflict serializable schedule. it does not cover the schedules Which are view serializable but not Conflict serializable schedules



Topic : Time stamp ordering protocols

Thomas-write time stamp ordering protocol :-

(or)

Time stamp ordering protocol with Thomas-write Rule :-

Thomas write Time Stamp ordering protocol Condition:

(i)

10 T_1	20 T_2
	R(A)
R(A)	

(ii)

10 T_1	20 T_2
	W(A)
R(A)	

(iii)

10 T_1	20 T_2
	R(A)
W(A)	

Same as B.T.S.O.P.

(iii)

10 T_1	20 T_2
	W(A)
W(A)	

It will be different with Thomas-write Rule

Thomas-Write Time Stamp ordering Protocol :

(i)

10	20
T_1	T_2
	R(A)
R(A)	

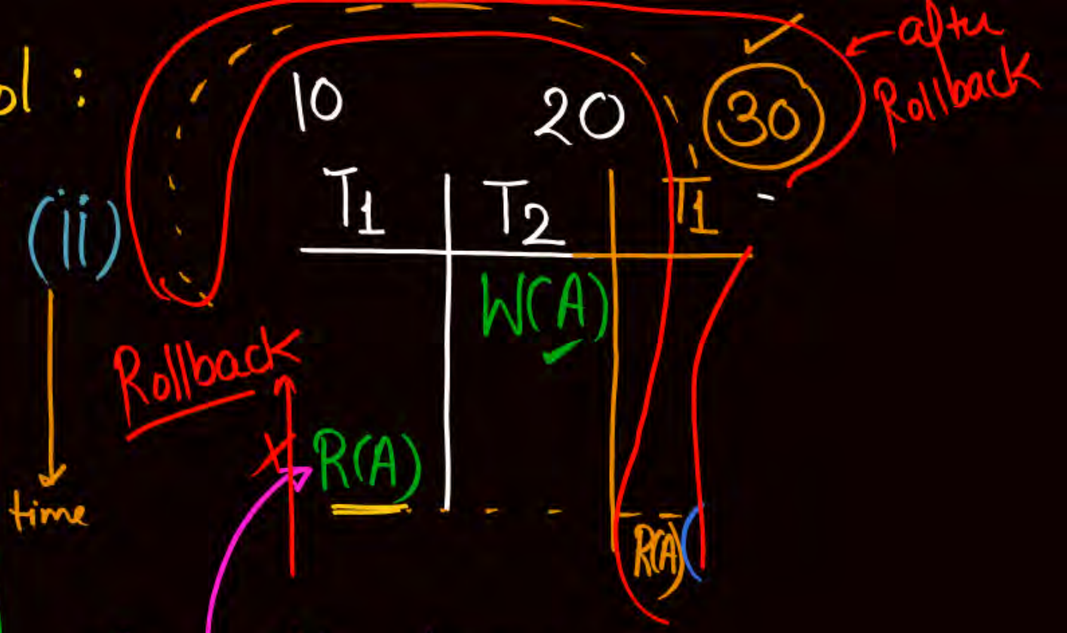
TS order = $T_1 \rightarrow T_2$

If T_1 performs this R(A) op^n at this point then also the schedule is view equivalent to serial schedule $T_1 \rightarrow T_2$ (Same as Time stamp order)

∴ Transaction T_1 is allowed to perform this R(A) op^n

And RTS(A) will remain same

(ii)



If we allow transaction T_1 to perform this R(A) op^n at this point then behaviour or schedule will not be view equivalent to serial schedule based on Time stamp order $T_1 \rightarrow T_2$

then T_1 is not allowed to perform this R(A) op^n using Thomas-write T.S.O.P.

∴ Rollback T_1

{ WTS(A) > TS(T_1) ∴ Rollback T_1 }

Thomas Write Time Stamp ordering Protocol :

(iii)

10	20
T_1	T_2
	R(A) ✓
W(A)	

TS order = $T_1 \rightarrow T_2$

Rollback

If T_1 performs this W(A), opn at this point then the schedule is not view equivalent to serial schedule $T_1 \rightarrow T_2$ (Time stamp order)

∴ T_1 is not allowed to perform this W(A) opn, ∴ Rollback T_1

If (RTS(A) > TS(T_1)) then Rollback T_1

(iv)

10	20
T_1	T_2
	W(A)
W(A)	

If transaction T_1 Performs this W(A) opn then behaviour of this schedule will not be view equivalent to serial schedule $T_1 \rightarrow T_2$ (based on time stamp order)

But if ignore this W(A) opn by transaction T_1 and continue with the other operations, then final value of dataitem 'A' in the database will be the value updated by T_2 ∴ it will become view equivalent to serial schedule ($T_1 \rightarrow T_2$)

∴ If (WTS(A) > TS(T_1)), then ignore this W(A) opn by transaction T_1 and Continue with other operations.

Thomas Write Time Stamp ordering Protocol Conditions :-

Let $TS(T_i) < TS(T_j)$ { i.e T_i is old & T_j is new
∴ Time stamp order = $T_i \rightarrow T_j$ }

Case (i) When transaction T_i issue a Read(A) op^n :-

ⓐ If $(WTS(A) > TS(T_i))$, then T_i is not allowed to perform R(A) op^n ∴ Rollback T_i ✓

ⓑ otherwise, { i.e. if $(WTS(A) \leq TS(T_i))$ }, then T_i is allowed to perform R(A) op^n,
∴ Perform R(A) op^n of transaction T_i
And set $RTS(A) = Max(RTS(A), TS(T_i))$

Thomas Write Time Stamp ordering Protocol Conditions :-

Let $TS(T_i) < TS(T_j)$ { i.e T_i is old & T_j is new
∴ Time stamp order = $T_i \rightarrow T_j$ }

Case (ii) When transaction T_i issue a Write(A) op^n :-

ⓐ If $(RTS(A) > TS(T_i))$, then T_i is not allowed to perform W(A) op^n ∴ Rollback T_i

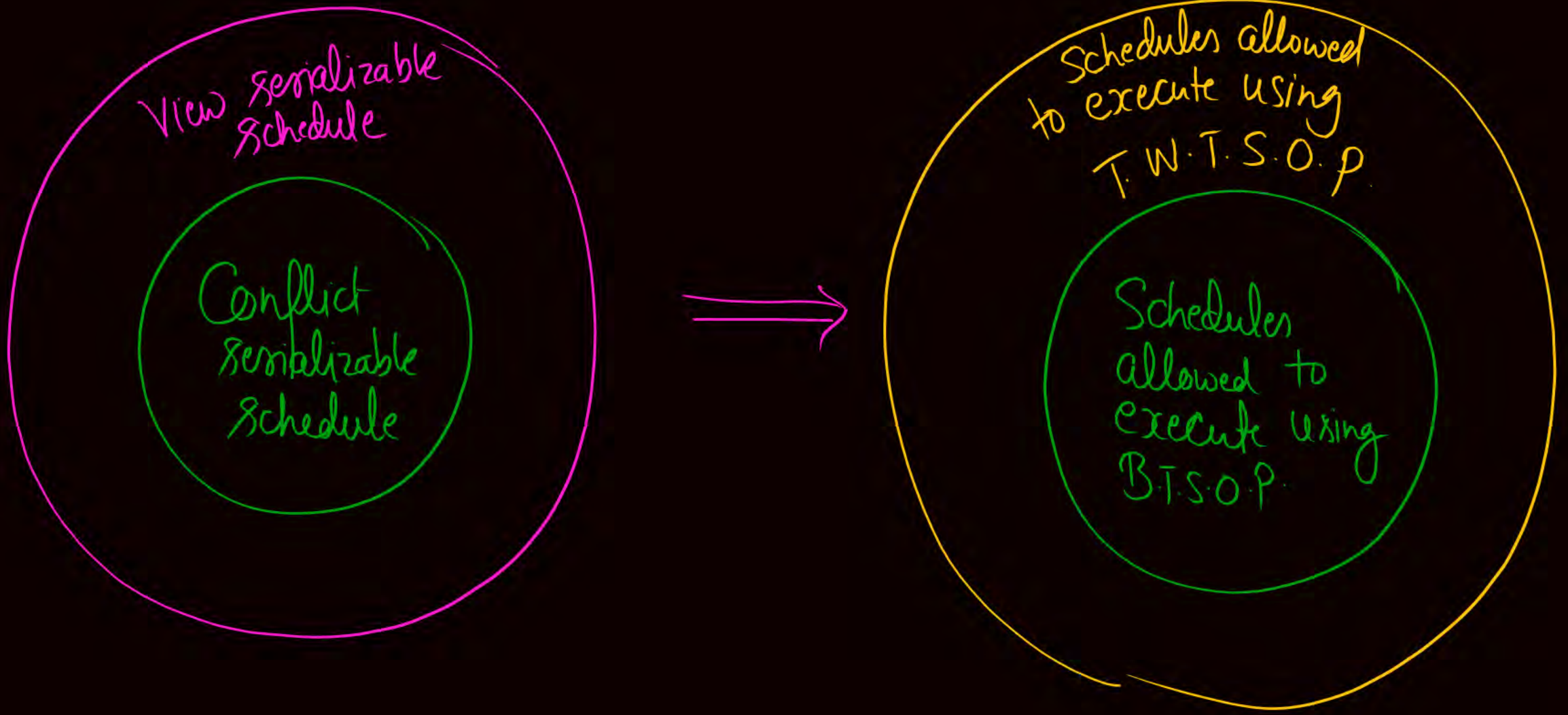
ⓑ If $(WTS(A) > TS(T_i))$, then ignore (&skip) it W(A) op^n by transaction T_i and Continue with the other operations.

③ Otherwise, T_i is allowed to perform W(A) op^n
∴ Perform this W(A) op^n by transaction T_i
and Set $WTS(A) = TS(T_i)$

Note: ① A schedule is allowed to execute using Thomas write time stamp ordering protocol only if schedule is view serializable schedule and view equivalent serial schedule is based on time stamp ordering of the transactions.

② If schedule is view serializable schedule but view equivalent serial schedule is not based on time stamp ordering of the transactions, then schedule is not allowed to execute using Tomas-write time stamp ordering protocol.

③ If schedule is not view serializable schedule, then it is never allowed to execute using T.W.T.S.O.P. for any time stamp ordering of transactions.


View serializable schedule
Conflict serializable schedule
Schedules allowed to execute using T.W.T.S.O.P.
Schedules allowed to execute using B.T.S.O.P.

B.T.S.O.P & T.W.T.S.O.P :-

1. May suffer from irrecoverability, Cascading Rollback, and lost update problem
2. B.T.S.O.P & T.W.T.S.O.P are free from dead lock but starvation is still possible.

→ To avoid this problem we can use strict time stamp ordering Protocol.

Strict time stamp ordering Protocol = Time Stamp ordering Protocol + Strict Recoverability Condition

{ To implement it, a transaction is allowed to R(A) or W(A) only if WTS(A) = 0



Topic : Deadlock ~~avoidance~~ Prevention algorithms

(For lock based Protocols)

* Concept of time-stamp assigned by DBMS can be used to ~~avoid~~ prevention the dead lock in lock based Protocols.

If dependency graph is cyclic, then deadlock may be possible

⇒

Dependency graph



it represent that T_1 requires the resource held by T_2

⇒

Dependency graph



Cyclic dependency graph
∴ Deadlock is possible

⇒

If we can Prevent from the cycles to exist in the dependency graph then Deadlock is not possible



Topic : Deadlock ~~avoidance~~ Prevention algorithms

(For lock based Protocols)

- There are two algorithms to prevent from deadlock to occur in the lock based protocols.

(i) Wait-die algorithm (Suicide)

(ii) Wound-wait algorithm

(Old - Young Concept)

Shareef insaan

Badmaash insaan

Kill

Old transactions will be preferred over young transactions.

i.e. If at all rollbacking is required then we will rollback new transaction



old will — young will

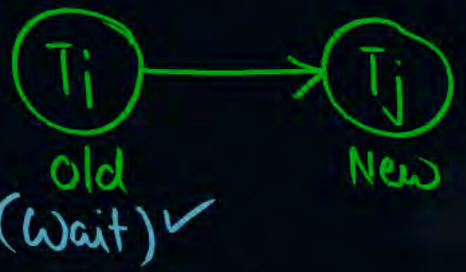
Topic : Wait-die algorithms

Let T_i & T_j are two transactions such that

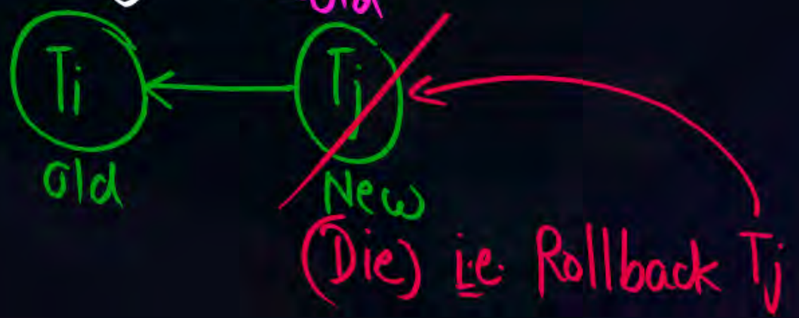
$TS(T_i) < TS(T_j)$

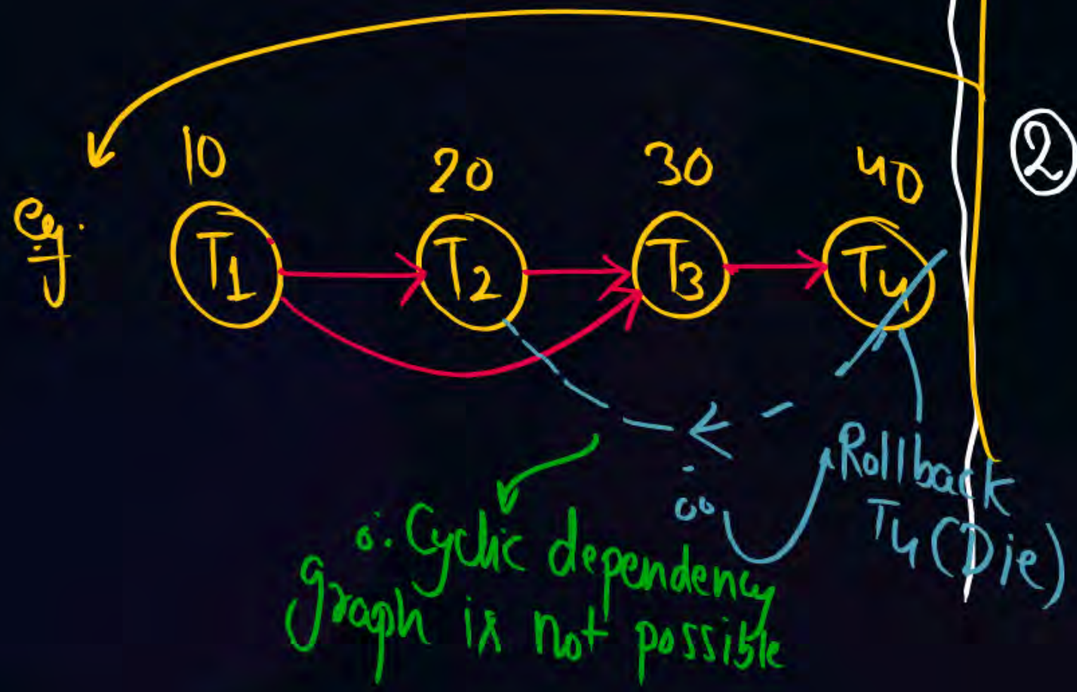
i.e. T_i (Old) T_j (New)

① If T_i (old) requires resource held by T_j (New) then T_i is allowed to wait



② If transaction T_j (young) requires the resource held by T_i (old)





Topic : Wound-wait algorithms

Old one will wound

Young one will wait

Let T_i & T_j are two transactions such that $TS(T_i) < TS(T_j)$



eg:



Again No cycle

∴ No deadlock

T2 will wound T4

Rollback T4

old

① If T_i requires the resource held by T_j

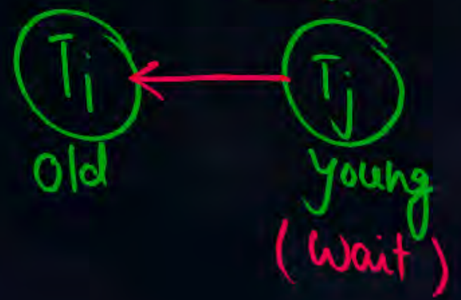


New

{ Wound T_j } → i.e. Rollback T_j

② If T_j requires the resource held by transaction T_i, then T_j is allowed to wait

old





2 mins Summary

- **Topic** Basic time stamp ordering protocol
- **Topic** Thomas write time stamp ordering protocol
- **Topic** Wait-die protocol for deadlock avoidance
- **Topic** Wound-wait protocol for deadlock avoidance

THANK - YOU

Slide